

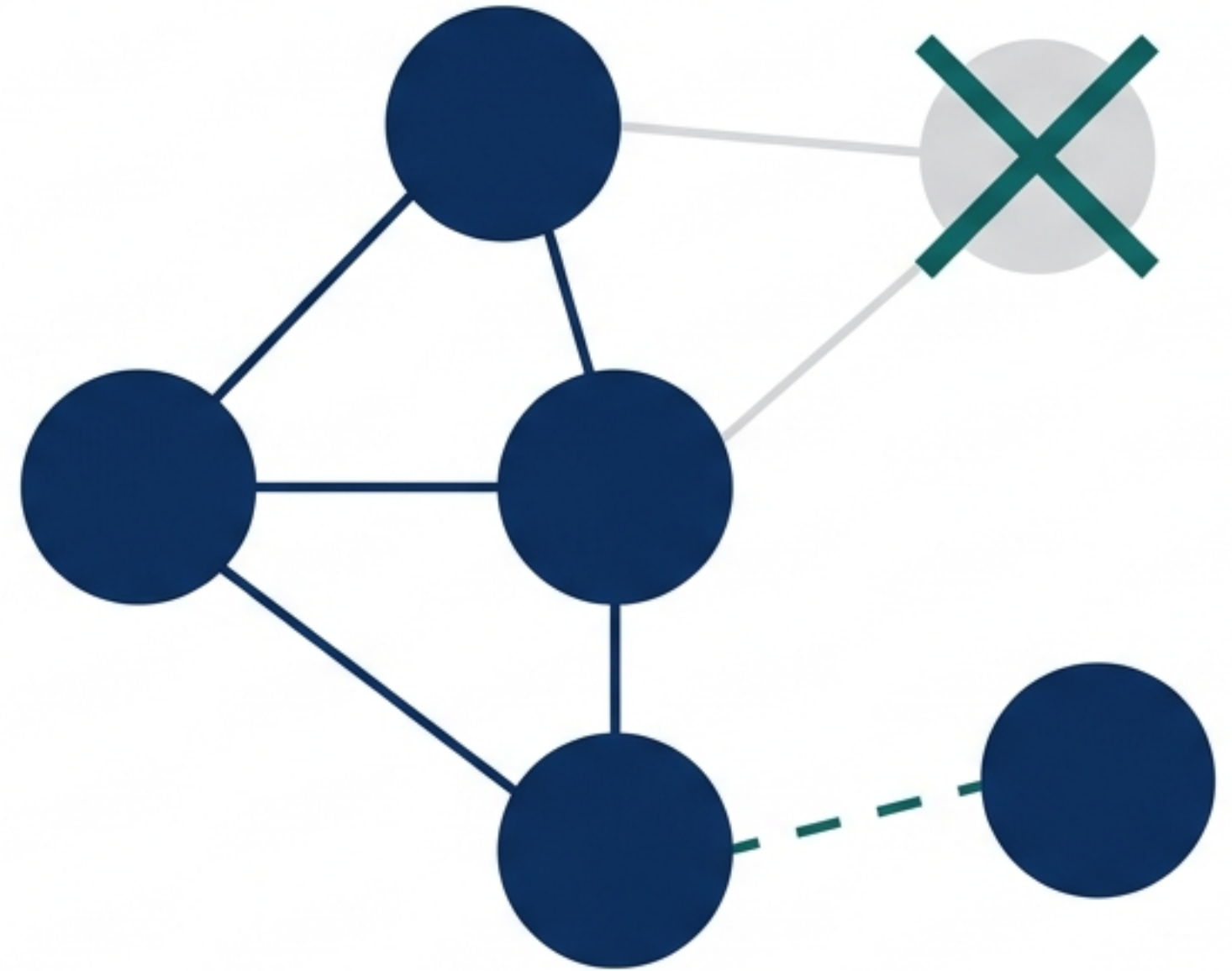
Failure Models in Distributed Systems

Understanding partial failures, system limits, and the impossibility of perfect agreement.



The Reality of Partial Failure

- In single machines, failures are total.
- In distributed systems, failures are **partial**.
- Some nodes keep working while others crash cleanly or act unpredictably.



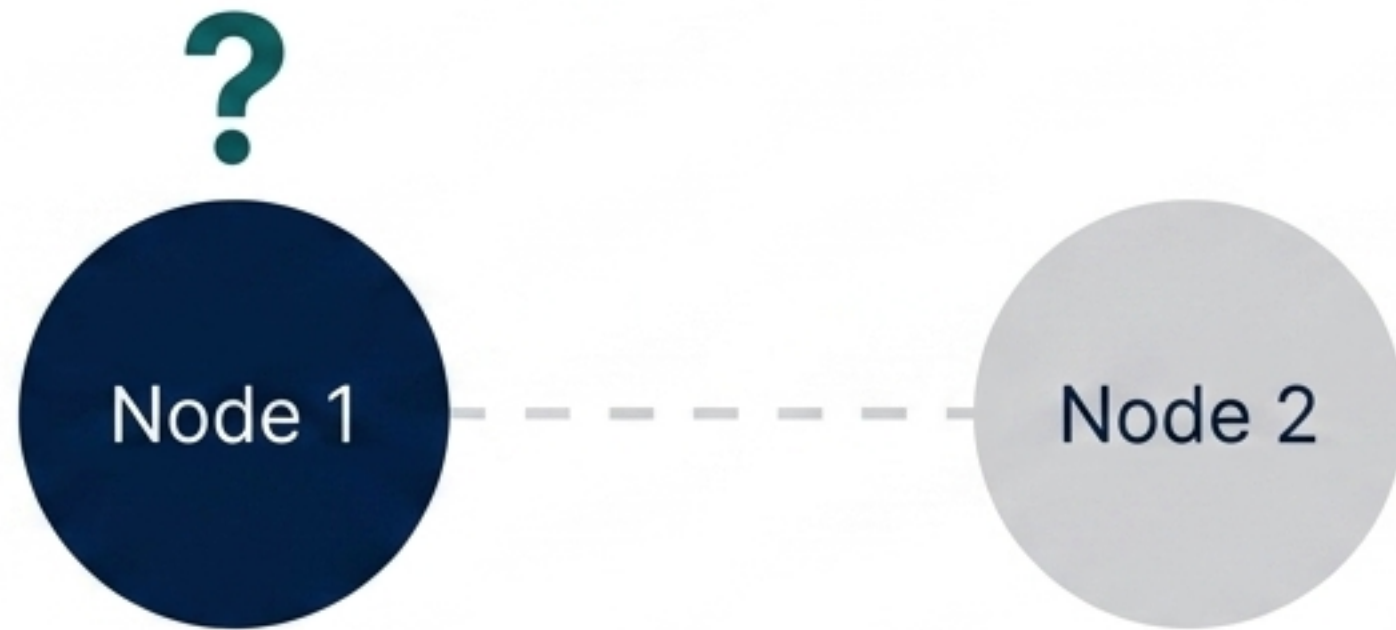
Categorizing System Failures

	Process (Nodes)	Channel (Network)
Benign (Clean / Omissions)	Crash, Fail-Stop, Omission	Message Drops, Network Delays
Arbitrary (Malicious / Corrupt)	Byzantine Behavior, Unintended Steps	Forged, Corrupted, or Duplicated Messages

Most distributed failures are benign. Arbitrary failures require the most complex cryptographic defenses.

Crash vs. Fail-Stop Models

Crash Failure



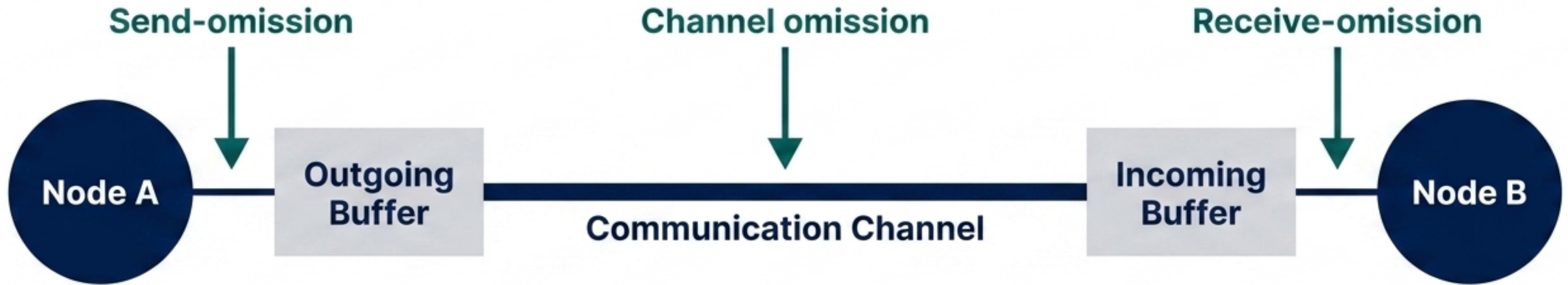
Process halts cleanly but remains silent. The network cannot confirm if it is dead or simply slow.

Fail-Stop Failure



Process halts cleanly, and other nodes can reliably detect the crash via guaranteed timeouts.

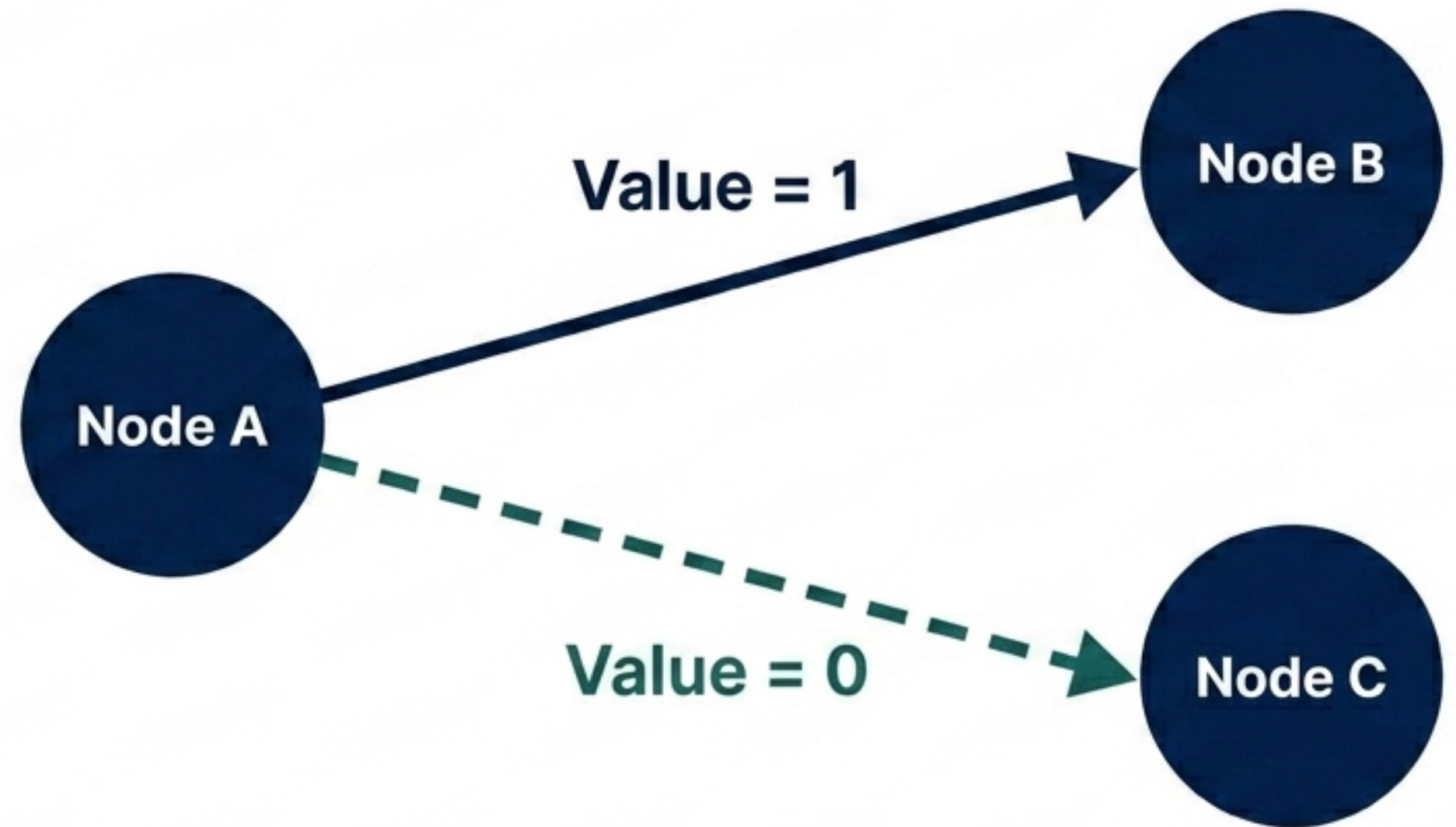
Omission: Where Messages Disappear



An omission failure occurs whenever a process or channel fails to perform its expected action.

The Worst-Case: Arbitrary (Byzantine)

- Nodes omit intended steps or take unintended ones.
- Data values are silently corrupted.
- Network channels duplicate or fabricate nonexistent messages.



Cannot be solved by simple timeouts. Demands checksums, sequence numbers, and consensus protocols.

Timing Failures (Synchronous Only)

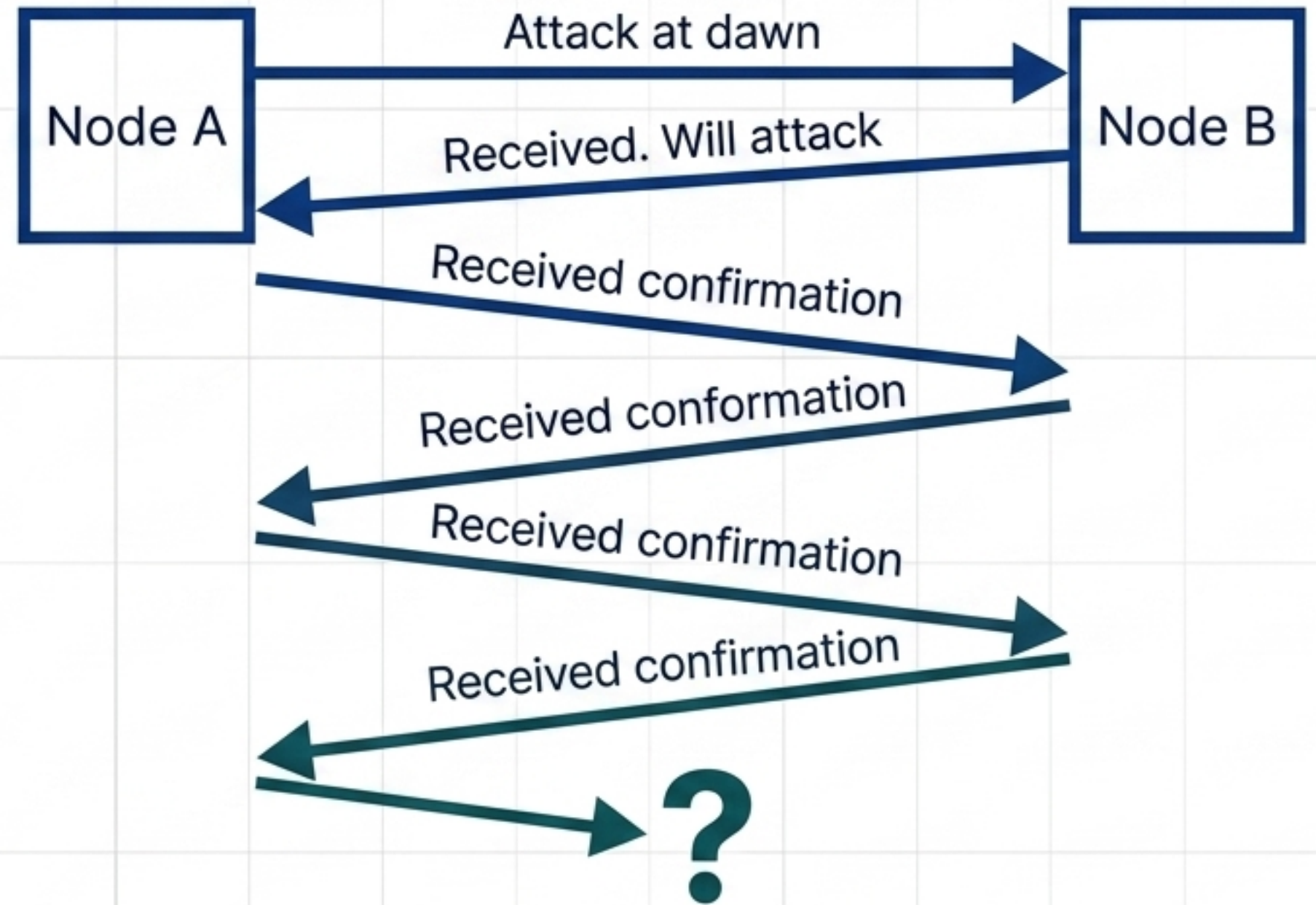
- **Clock drift:** Local clock bounds exceed guaranteed real-time drift rates.
- **Performance:** Processing steps or message delivery exceeds strict time limits.



In asynchronous networks, slow servers are not failing—they are just slow. Timing failures require strict systemic guarantees.

The Impossibility of Agreement

Two armies (Pepperland divisions) must attack simultaneously. They communicate via messengers across a valley where any messenger can be captured.



The Asynchronous Dilemma

**Is the node dead, or is
the network just slow?**

Because perfect clocks and guaranteed delivery do not exist in asynchronous systems, perfect failure detection is mathematically impossible.

How We Survive: Masking Failures

We build reliable services on top of unreliable components by hiding failures or converting them into safer types.



Retransmission

Masks channel omission failures by resending lost packets.



Checksums & Sequence Numbers

Converts Byzantine corruption into simple omission failures by discarding altered messages.



Replication

Masks complete process crashes by failing over to identical, redundant servers.

The Engineering Reality

- We cannot achieve perfect failure detection in the real world.
- Therefore, modern systems compromise: we rely heavily on **Timeouts**.
- We design assuming a **Crash-Stop** or **Fail-Stop** model, accepting that we will occasionally guess wrong.

We treat a highly delayed node as a dead node, and engineer the system to recover gracefully.

Key Architectural Takeaways

- **Partial Failure is Default:** Systems must be designed to outlive the failure of their individual components.
- **Know Your Taxonomy:** Crash, Omission, Timing, and Byzantine semantics dictate the defensive architecture required.
- **Accept the Limits:** The Two Generals problem proves that absolute certainty over unreliable channels is impossible.
- **Design for Recovery:** Rely on masking, replication, and fail-stop assumptions to engineer practical, real-world reliability.